

Lecture # 05

Multi-Threading

PARALLEL AND DISTRIBUTED COMPUTING

CS-482



Multithreading in Operating System

Multithreading is a feature in operating systems that allows a program to do several tasks at the same time. Think of it like having multiple hands working together to complete different parts of a job faster. Each “hand” is called a thread, and they help make programs run more efficiently. Multithreading makes your computer work better by using its resources more effectively, leading to quicker and smoother performance for applications like web browsers, games, and many other programs you use every day.

How Does Multithreading Work?

Multithreading works by allowing a computer's processor to handle multiple tasks at the same time. Even though the processor can only do one thing at a time, it switches between different threads from various programs so quickly that it looks like everything is happening all at once.

Here's how it simplifies:

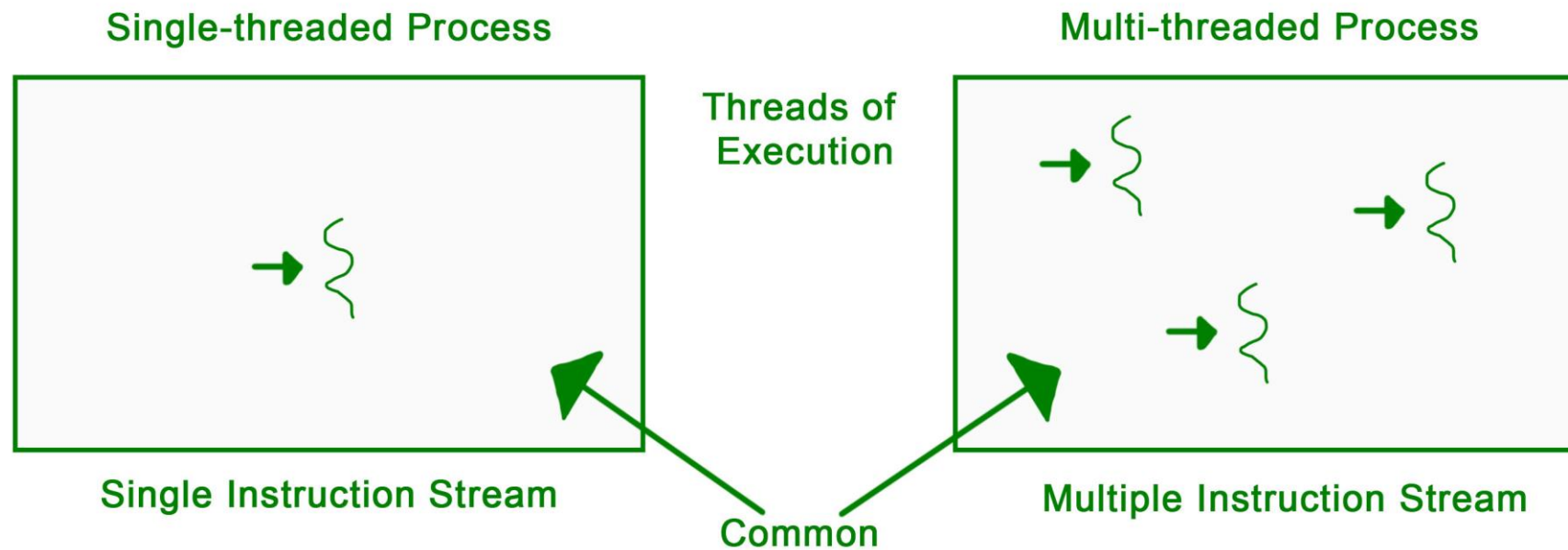
- **Processor Handling** : The processor can execute only one instruction at a time, but it switches between different threads so fast that it gives the illusion of simultaneous execution.
- **Thread Synchronization** : Each thread is like a separate task within a program. They share resources and work together smoothly, ensuring programs run efficiently.
- **Efficient Execution** : Threads in a program can run independently or wait for their turn to process, making programs faster and more responsive.
- **Programming Considerations** : Programmers need to be careful about managing threads to avoid problems like conflicts or situations where threads get stuck waiting for each other.

What is Multitasking?

Multitasking is the ability of an operating system to run multiple programs or tasks at the same time. It allows you to perform different activities simultaneously on your computer. For example, you can listen to music while browsing the internet and typing a document all at once.

Multitasking is of two types: Processor-based and thread-based. Processor-based multitasking is managed by the OS, however, multitasking through multithreading can be controlled by the programmer to some extent. The concept of **multi-threading** needs a proper understanding of these two terms – **a process and a thread**. A process is a program being executed. A process can be further divided into independent units known as threads. A thread is like a small light-weight process within a process. Or we can say a collection of threads is what is known as a process.

What is Multitasking?



Single Thread and Multi Thread Process

Applications:

Threading is used widely in almost every field. Most widely it is seen over the internet nowadays where we are using transaction processing of every type like recharges, online transfer, banking etc. Threading is a segment which divide the code into small parts that are of very light weight and has less burden on CPU memory so that it can be easily worked out and can achieve goal in desired field. The concept of threading is designed due to the problem of fast and regular changes in technology and less the work in different areas due to less application. Then as says “need is the generation of creation or innovation” hence by following this approach human mind develop the concept of thread to enhance the capability of programming.

Multithreading vs Multitasking

Feature	Multithreading	Multitasking
Definition	Running multiple threads within a single program simultaneously.	Running multiple programs or tasks concurrently.
Example	Web browser loading a page, handling user input, and downloading files simultaneously.	Listening to music, browsing the web, and typing a document at the same time.
Scope	Within a single program.	Across multiple programs.
Resource Use	Utilizes CPU resources more efficiently within a program.	Manages system resources to allocate time and memory to different programs.
Purpose	Enhances the performance and responsiveness of a single application.	Improves overall system efficiency by allowing concurrent execution of multiple programs.
Switching	Threads are managed by the program itself.	Programs are managed by the operating system, which switches between them.

Lifecycle of a Thread

There are various stages in the lifecycle of a thread. Following are the stages a thread goes through in its whole life.

- **New:** The lifecycle of a born thread (new thread) starts in this state. It remains in this state till a program starts.
- **Runnable :** A thread becomes runnable after it starts. It is considered to be executing the task given to it.
- **Waiting :** While waiting for another thread to perform a task, the currently running thread goes into the waiting state and then transitions back again after receiving a signal from the other thread.
- **Timed Waiting:** A runnable thread enters into this state for a specific time interval and then transitions back when the time interval expires or the event the thread was waiting for occurs.
- **Terminated (Dead) :** A thread enters into this state after completing its task.

Types of Execution in OS

There are two types of execution:

- **Concurrent Execution:** This occurs when a processor is successful in switching resources between threads in a multithreaded process on a single processor.
- **Parallel Execution:** This occurs when every thread in the process runs on a separate processor at the same time and in the same multithreaded process

Drawbacks of Multithreading

Multithreading is complex and many times difficult to handle. It has a few drawbacks. These are:

- If you don't make use of the locking mechanisms properly, while investigating data access issues there is a chance of problems arising like data inconsistency and dead-lock.
- If many threads try to access the same data, then there is a chance that the situation of thread starvation may arise. Resource contention issues are another problem that can trouble the user.
- Display issues may occur if threads lack coordination when displaying data.

Benefits of Multithreading

- Multithreading can improve the performance and efficiency of a program by utilizing the available CPU resources more effectively. Executing multiple threads concurrently, it can take advantage of parallelism and reduce overall execution time.
- Multithreading can enhance responsiveness in applications that involve user interaction. By separating time-consuming tasks from the main thread, the user interface can remain responsive and not freeze or become unresponsive.
- Multithreading can enable better resource utilization. For example, in a server application, multiple threads can handle incoming client requests simultaneously, allowing the server to serve more clients concurrently.
- Multithreading can facilitate better code organization and modularity by dividing complex tasks into smaller, manageable units of execution. Each thread can handle a specific part of the task, making the code easier to understand and maintain.

What is the difference between a thread and a process?.

A process is a complete program in execution, while a thread is a smaller part of a process that can run independently.

Introduction of Process Synchronization

Process Synchronization is used in a computer system to ensure that multiple processes or threads can run concurrently without interfering with each other.

The main objective of process synchronization is to ensure that multiple processes access shared resources without interfering with each other and to prevent the possibility of inconsistent data due to concurrent access. To achieve this, various synchronization techniques such as semaphores, monitors, and critical sections are used.

In a multi-process system, synchronization is necessary to ensure data consistency and integrity, and to avoid the risk of deadlocks and other synchronization problems. Process synchronization is an important aspect of modern operating systems, and it plays a crucial role in ensuring the correct and efficient functioning of multi-process systems.

On the basis of synchronization, processes are categorized as one of the following two types:

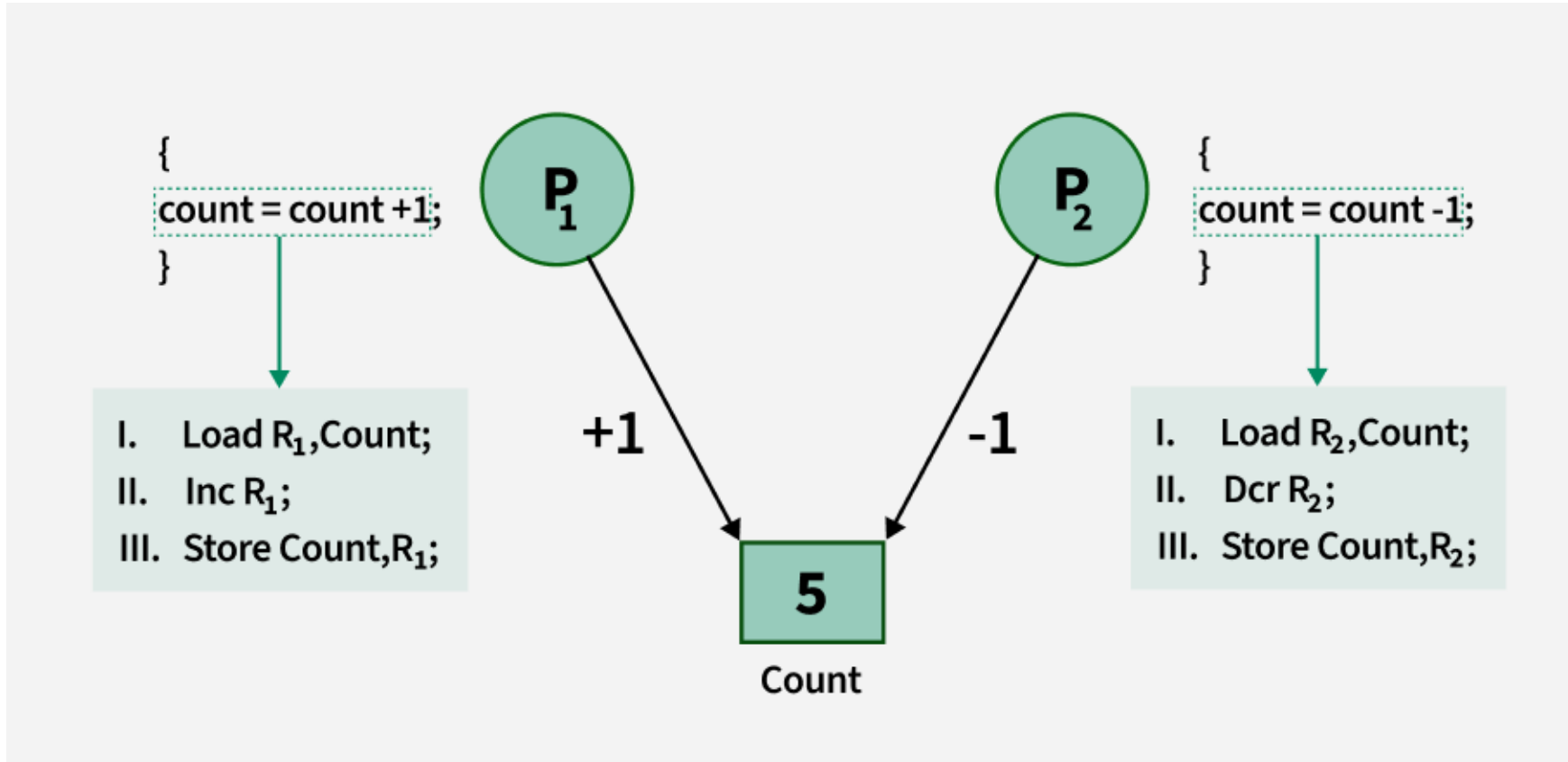
- **Independent Process:** The execution of one process does not affect the execution of other processes.
- **Cooperative Process:** A process that can affect or be affected by other processes executing in the system.

Process synchronization problem arises in the case of Cooperative processes also because resources are shared in Cooperative processes.

Process Synchronization

Process Synchronization is the coordination of execution of multiple processes in a multi-process system to ensure that they access shared resources in a controlled and predictable manner. It aims to resolve the problem of race conditions and other synchronization issues in a concurrent system.

Process Synchronization



Process Synchronization

Lack of Synchronization in Inter Process Communication Environment leads to following problems:

- 1.Inconsistency:** When two or more processes access shared data at the same time without proper synchronization. This can lead to conflicting changes, where one process's update is overwritten by another, causing the data to become unreliable and incorrect.
- 2.Loss of Data:** Loss of data occurs when multiple processes try to write or modify the same shared resource without coordination. If one process overwrites the data before another process finishes, important information can be lost, leading to incomplete or corrupted data.
- 3.Deadlock:** Lack of Synchronization leads to Deadlock which means that two or more processes get stuck, each waiting for the other to release a resource. Because none of the processes can continue, the system becomes unresponsive and none of the processes can complete their tasks.

Types of Process Synchronization

The two primary type of process Synchronization in an Operating System are:

1.Competitive: Two or more processes are said to be in Competitive Synchronization if and only if they compete for the accessibility of a shared resource.

Lack of Synchronization among Competing process may lead to either Inconsistency or Data loss.

2.Cooperative: Two or more processes are said to be in Cooperative Synchronization if and only if they get affected by each other i.e. execution of one process affects the other process.

Lack of Synchronization among Cooperating process may lead to Deadlock.

Conditions That Require Process Synchronization

- 1.Critical Section:** It is that part of the program where shared resources are accessed. Only one process can execute the critical section at a given point of time. If there are no shared resources, then no need of synchronization mechanisms.
- 2.Race Condition:** It is a situation wherein processes are trying to access the critical section and the final result depends on the order in which they finish their update. Process Synchronization mechanism need to ensure that instructions are being executed in a required order only.
- 3.Pre-Emption:** Preemption is when the operating system stops a running process to give the CPU to another process. This allows the system to make sure that important tasks get enough CPU time. This is important as mainly issues arise when a process has not finished its job on shared resource and got preempted. The other process might end up reading an inconsistent value if process synchronization is not done.

What is Race Condition?

A race condition is a situation that may occur inside a critical section. This happens when the result of multiple process/thread execution in the critical section differs according to the order in which the threads execute. Race conditions in critical sections can be avoided if the critical section is treated as an atomic instruction. Also, proper thread synchronization using locks or atomic variables can prevent race conditions.

Let us consider the following example.

- There is a shared variable balance with value 100.
- There are two processes deposit(10) and withdraw(10). The deposit process does $\text{balance} = \text{balance} + 10$ and withdraw process does $\text{balance} = \text{balance} - 10$.
- Suppose these processes run in an interleaved manner. The deposit() fetches the balance as 100, then gets preempted.
- Now withdraw() get scheduled and makes balance 90.
- Finally, deposit is rescheduled and makes the value as 110. This value is not correct as the balance after both operations should be 100 only

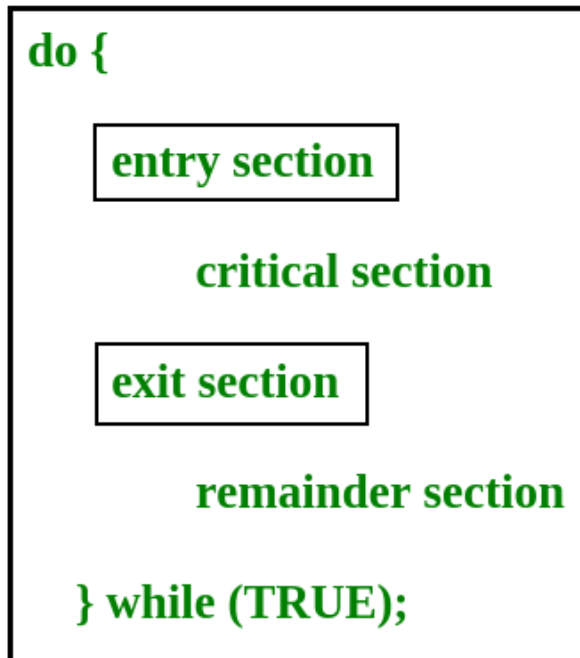
We can not notice that the different segments of two processes running in different order would give different values of balance.

Critical Section Problem

A critical section is a code segment that can be accessed by only one process at a time. The critical section contains shared variables that need to be synchronized to maintain the consistency of data variables. So, the critical section problem means designing a way for cooperative processes to access shared resources without creating data inconsistencies.

In the above example, the operations that involve balance variable should be put in critical sections of both deposit and withdraw.

Critical Section Problem



Classical IPC Problems

Various classical Inter-Process Communication (IPC) problems include:

- Producer Consumer Problem
- Readers-Writers Problem
- Dining Philosophers Problem

Producer Consumer Problem

The Producer-Consumer Problem is a classic example of process synchronization. It describes a situation where two types of processes producers and consumers share a common, limited-size buffer or storage.

- **Producer:** A producer creates or generates data and puts it into the shared buffer. It continues to produce data as long as there is space in the buffer.
- **Consumer:** A consumer takes data from the buffer and uses it. It continues to consume data as long as there is data available in the buffer.

The challenge arises because both the producer and the consumer need to access the same buffer at the same time, and if they do not properly synchronize their actions, issues can occur.

Producer Consumer Problem

- 1. Buffer Overflow:** If the producer tries to add data when the buffer is full, there will be no space for new data, causing the producer to be blocked.
- 2. Buffer Underflow:** If the consumer tries to consume data when the buffer is empty, it has nothing to consume, causing the consumer to be blocked.

Readers-Writers Problem

The Readers-Writers Problem is a classic synchronization problem where multiple processes are involved in reading and writing data from a shared resource. This problem typically involves two types of processes:

- **Readers:** These processes only read data from the shared resource and do not modify it.
- **Writers:** These processes modify or write data to the shared resource.

Readers-Writers Problem

The challenge in the Reader-Writer problem is to allow multiple readers to access the shared data simultaneously without causing issues. However, only **one writer** should be allowed to write at a time, and no reader should be allowed to read while a writer is writing. This ensures the integrity and consistency of the data.

Dining Philosophers Problem

The **Dining Philosophers Problem** is a well-known example that shows the difficulties of sharing resources and preventing deadlock when multiple processes are involved. The problem involves a set of philosophers sitting around a dining table. Each philosopher thinks deeply, but when they are hungry, they need to eat. However, to eat, they must pick up two forks from the table, one from the left and one from the right.

Problem Setup:

- There are **five philosophers** sitting around a circular table.
- Each philosopher has a plate of food in front of them and a fork to their left and right.
- A philosopher needs both forks to eat. If they pick up a fork, they hold it until they finish eating.
- After eating, they put down both forks and start thinking again.

The problem arises when multiple philosophers try to pick up forks at the same time, which can lead to a situation where each philosopher holds one fork but cannot get the second fork, leading to a **deadlock**. Additionally, there's a risk of **starvation** if some philosophers are continually denied the opportunity to eat.

Advantages of Process Synchronization

- Ensures data consistency and integrity
- Avoids race conditions
- Prevents inconsistent data due to concurrent access
- Supports efficient and effective use of shared resources

Disadvantages of Process Synchronization

- Adds overhead to the system
- This can lead to performance degradation
- Increases the complexity of the system
- Can cause deadlock if not implemented properly.

Process synchronization solution

Concurrent computing requires process synchronization to coordinate numerous processes in a multi-process system to regulate and forecast resource access. It addresses race situations and data inconsistency, essential for data integrity. Semaphores and Peterson's solution are used for synchronization. Synchronization is necessary for data consistency but adds complexity and performance overheads, making correct implementation and management vital for multi-process systems.

Critical Section in Synchronization

A **critical section** is a part of a program where shared resources like memory or files are accessed by multiple processes or threads. To avoid issues like data inconsistency or race conditions, synchronization techniques ensure that only one process or thread uses the critical section at a time.

- The critical section contains shared variables or resources that need to be synchronized to maintain the consistency of data variables.
- In simple terms, a critical section is a group of instructions/statements or regions of code that need to be executed atomically, such as accessing a resource (file, input or output port, global data, etc.) In concurrent programming, if one process tries to change the value of shared data at the same time as another thread tries to read the value (i.e., data race across threads), the result is unpredictable. The access to such shared variables (shared memory, shared files, shared port, etc.) is to be synchronized.

Properties that should be followed if any code in the critical section

Few programming languages have built-in support for synchronization. It is critical to understand the importance of race conditions while writing kernel-mode programming (a device driver, kernel thread, etc.) since the programmer can directly access and modify kernel data structures

Although there are some **properties that should be followed if any code in the critical section**

- 1. Mutual Exclusion:** If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
- 2. Progress:** If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely.
- 3. Bounded Waiting:** There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Critical Section in Synchronization

Two general approaches are used to handle critical sections:

- 1. Preemptive kernels:** A preemptive kernel allows a process to be preempted while it is running in kernel mode.
- 2. Non preemptive kernels:** A non-preemptive kernel does not allow a process running in kernel mode to be preempted. A kernel-mode process will run until it exists in kernel mode, blocks, or voluntarily yields control of the CPU. A non-preemptive kernel is essentially free from race conditions on kernel data structures, as only one process is active in the kernel at a time.

Critical Section Problem

The use of critical sections in a program can cause a number of issues, including:

- **Deadlock:** When two or more threads or processes wait for each other to release a critical section, it can result in a deadlock situation in which none of the threads or processes can move. Deadlocks can be difficult to detect and resolve, and they can have a significant impact on a program's performance and reliability.
- **Starvation:** When a thread or process is repeatedly prevented from entering a critical section, it can result in starvation, in which the thread or process is unable to progress. This can happen if the critical section is held for an unusually long period of time, or if a high-priority thread or process is always given priority when entering the critical section.
- **Overhead:** When using critical sections, threads or processes must acquire and release locks or semaphores, which can take time and resources. This may reduce the program's overall performance.

Critical Section Problem

Entry Section

Critical
Section

Exit Section

Remainder
Section

Critical Section Problem

It could be visualized using the pseudo-code below –

```
do{  
    flag=1;  
    while(flag); // (entry section)  
        // critical section  
    if (!flag)  
        // remainder section  
} while(true);
```

Solution to Critical Section Problem

A simple solution to the critical section can be thought of as shown below,

```
acquireLock();
```

```
Process Critical Section
```

```
releaseLock();
```

Advantages of Critical Section in Process Synchronization

- 1.Prevents race conditions:** By ensuring that only one process can execute the critical section at a time, race conditions are prevented, ensuring consistency of shared data.
- 2.Provides mutual exclusion:** Critical sections provide mutual exclusion to shared resources, preventing multiple processes from accessing the same resource simultaneously and causing synchronization-related issues.
- 3.Reduces CPU utilization:** By allowing processes to wait without wasting CPU cycles, critical sections can reduce CPU utilization, improving overall system efficiency.
- 4.Simplifies synchronization:** Critical sections simplify the synchronization of shared resources, as only one process can access the resource at a time, eliminating the need for more complex synchronization mechanisms.

Disadvantages of Critical Section in Process Synchronization

- 1.Overhead:** Implementing critical sections using synchronization mechanisms like semaphores and mutexes can introduce additional overhead, slowing down program execution.
- 2.Deadlocks:** Poorly implemented critical sections can lead to deadlocks, where multiple processes are waiting indefinitely for each other to release resources.
- 3.Can limit parallelism:** If critical sections are too large or are executed frequently, they can limit the degree of parallelism in a program, reducing its overall performance.
- 4.Can cause contention:** If multiple processes frequently access the same critical section, contention for the critical section can occur, reducing performance.

Important Points Related to Critical Section in Process Synchronization

1. Understanding the concept of critical section and why it's important for synchronization.
2. Familiarity with the different synchronization mechanisms used to implement critical sections, such as semaphores, mutexes, and monitors.
3. Knowledge of common synchronization problems that can arise in critical sections, such as race conditions, deadlocks, and live locks.
4. Understanding how to design and implement critical sections to ensure proper synchronization of shared resources and prevent synchronization-related issues.
5. Familiarity with best practices for using critical sections in concurrent programming.